Computer Architecture & Concurrency – COMP0008
Lecture 1: Introduction (Part 3)

Kevin Bryson
k.bryson@ucl.ac.uk
Darwin Building Room 632.

So to end with … the start …

Part I: Overview and motivation behind understanding computer architecture

Again this is an informal introduction to computer architecture ...

In later lectures we give a detailed knowledge of how a MIPS32 processor works, how to program it in assembly and machine code, and how parallelism is employed in the architecture to make it run faster.

When was the first ‘computer’ built by humans?

- Antikythera computer.

- Very basic instruction set architecture (fixed multiplication and division with fixed data flow between instructions ... no data registers to store values).
- If we were to design this ‘computer architecture’ ... we would probably have designed it’s *organization* out of electronic logic gates and programmed it using C ...
- Or we could have used Lego ...

But was this a computer?

It did compute with “input data” following an algorithm and produce “output data”.

But what were its limitations?
What is it missing?

Any ideas:

- General purpose?
- Programmable?
- Turing Complete? (Look this up ...)

Antikythera “computer”

Architecture

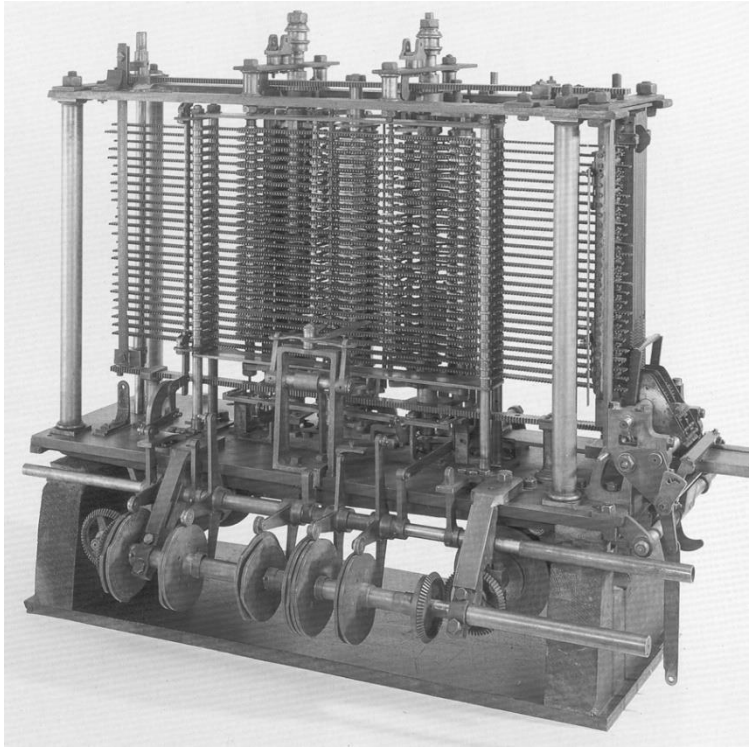
- Data values — *Continuous floating point values*
- Storage — Stored in degrees rotation of cogs.
- Operations — Divisions and multiplications
- Programmable. — Only by rebuilding machine.
- General purpose? — Not “Turing Complete” (Requires branching instructions)

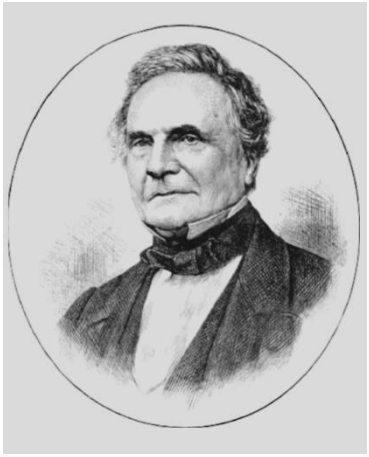
Organization

- Fixed layout of wheels and cogs to do calculations.

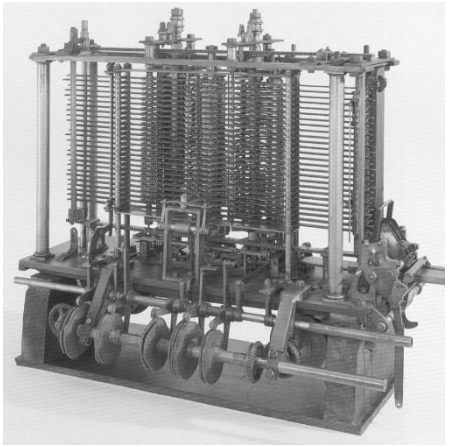
The Analytical Engine

- Designed by Charles Babbage from 1834-1871.
- Considered to be the first "*digital* computer" running a "program" from punched cards.
- Built of mechanical gears represented discrete value (0-9).
- All the key elements of a modern computer.





Babbage's Analytical Engine



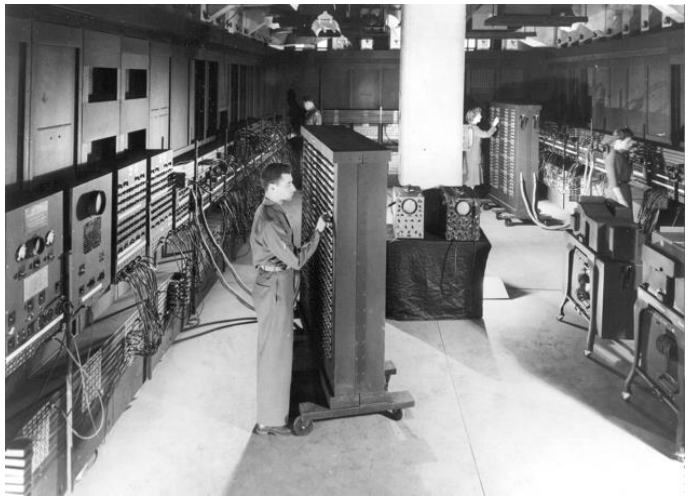
Architecture

- Data values — *Discrete 40 digit* ***decimal*** *values*
- Storage — Stored in *discrete* rotation of wheels (with a memory of 1000 40-digit values)
- Operations — +, -, *, /, comparisons, optional sqrt !
- Programmable. — Yes – using punched cards. (Similar in approach to modern day assembly language.)
- General purpose? — Yes - "Turing Complete" (Requires branching instructions)

Organization

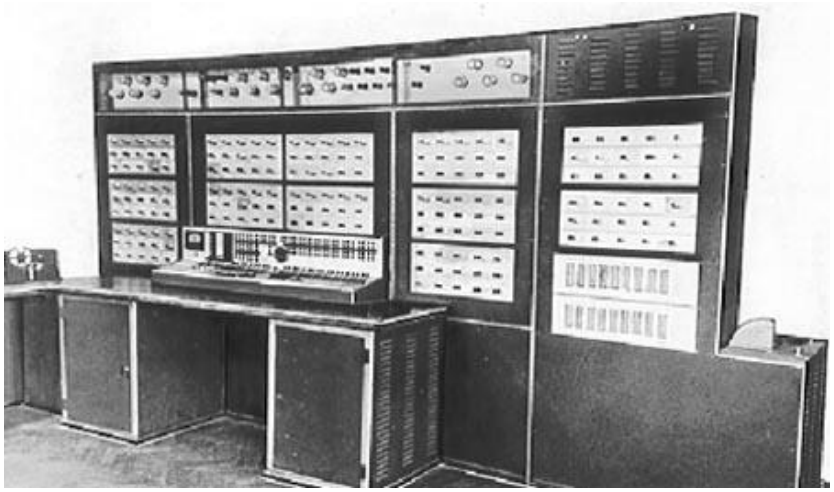
- Fixed layout of wheels and cogs to do calculations.

Experimental architectures ...
What number base should we be using?



ENIAC 1 used decimal calculations like the Analytical Engine.

- Russian scientists created a machine called Setun that used ternary (Base 3).
- It used negative, zero and positive voltages for the three symbols.



Setun (1958, Russia)

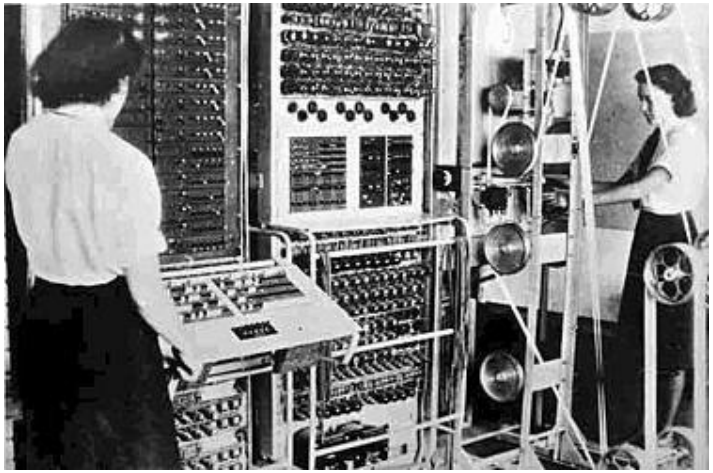
Anyone know what this is ?

Why use a discrete number of levels? More information in continuous voltages?

- The Antikythera computer (both real and in Lego).
- In a way this was an analogue mechanical computer using the ‘angle’ of different wheels between 0 and 360 degrees.

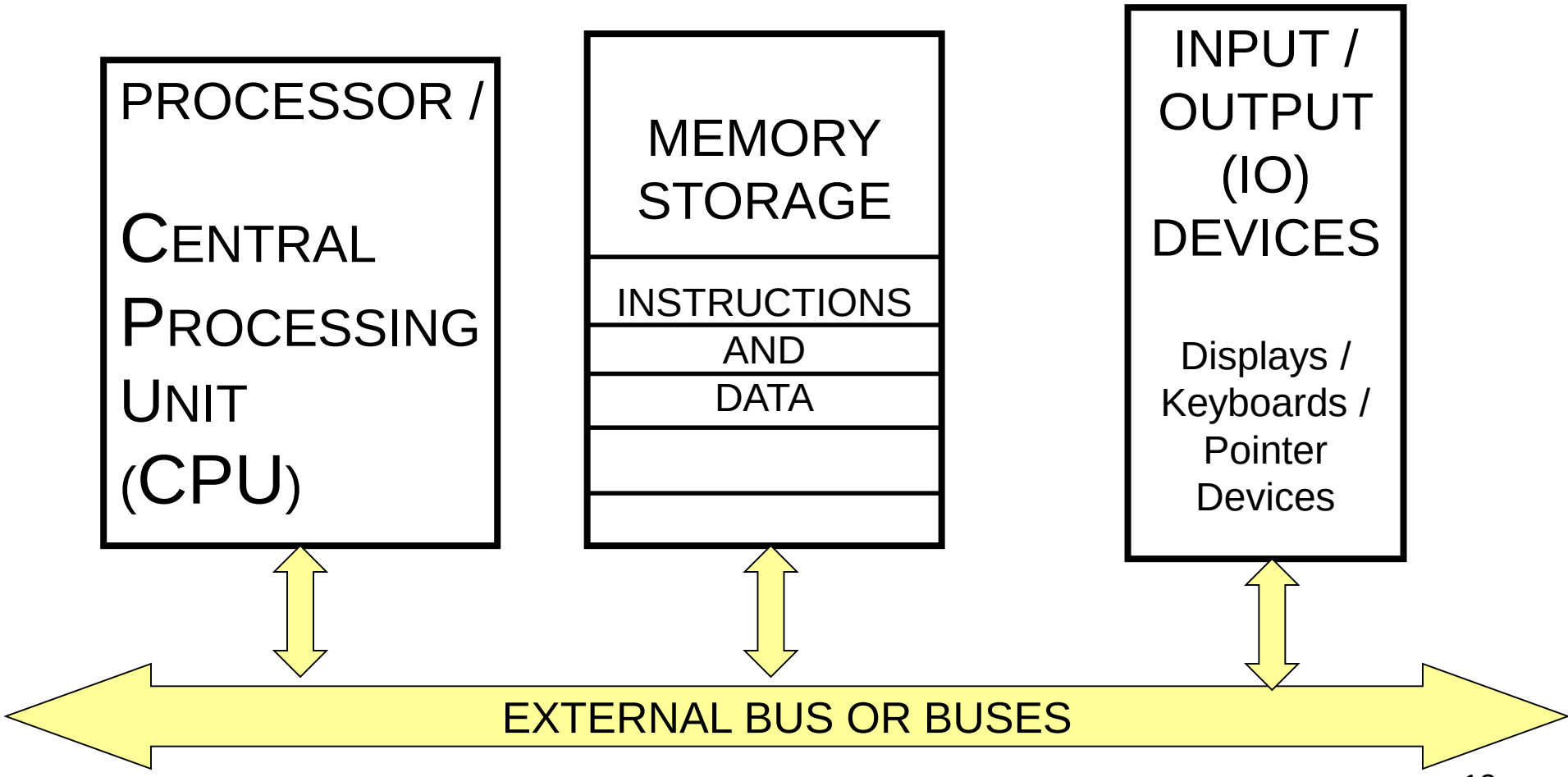
- Heathkit educational analogue computer used continuous voltage values …
- Essentially 1V + 1V = 2V although complex operations such as differentiation could be accomplished.

Colossus – binary programmable electronic computer

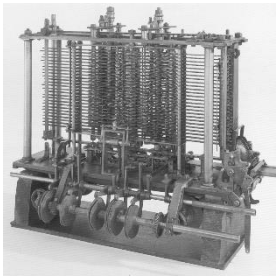


- Colossus was built in 1943 at Bletchley Park (just outside Cambridge) and used to help decrypt the German Lorenz Cipher.
- It is regarded as one of the first digital programmable electronic computers (**but again using switches and patched cables**).
- Its organization used thermionic valves (vacuum tubes) to carry out **binary operations (Boolean algebra)**.

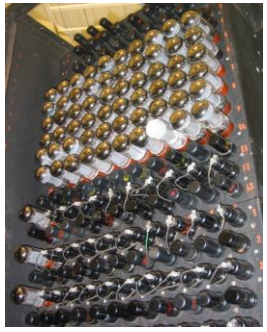
Computers settled into a “Stored program” von Neumann Architecture



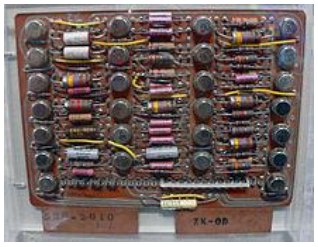
But the computer architecture / hardware radically changed over the years ...



The first “von Neumann architecture” and Turing-complete computer
Analytical Engine
Mechanical gears

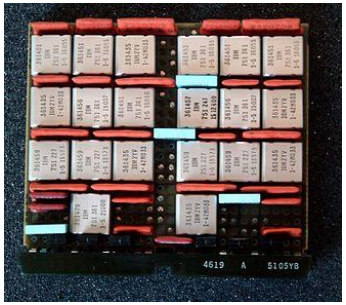


One of the first electronic computers
ENIAC 1
Thermionic valves

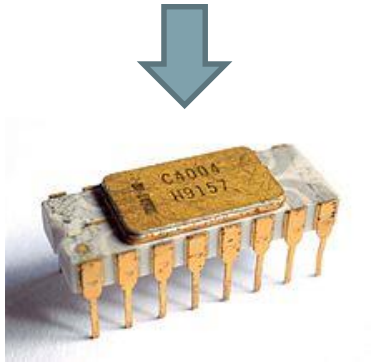


One of the first transistor electronic computers
IBM 7030
Individual transistors

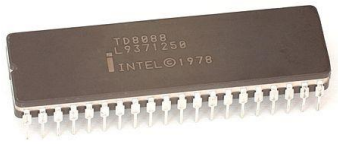
But the computer architecture / hardware radically changed over the years ...



Discrete chips with transistors
IBM 360
Discrete chips with logic gates.

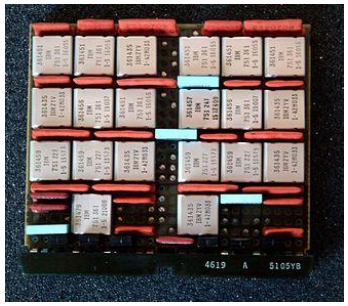


Intel 4004
Busicom 141-PF
First microprocessor on a chip.



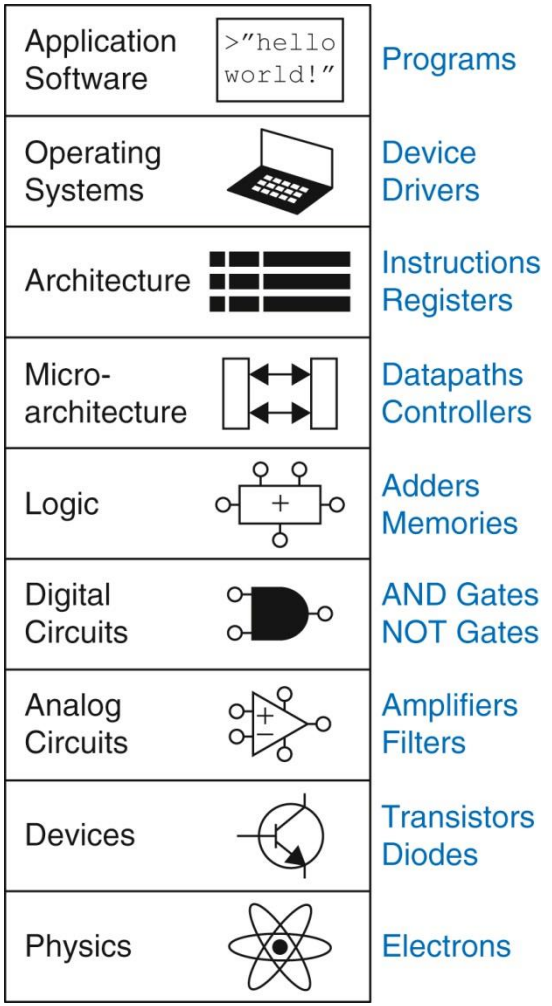
Intel 8088 16-bit processor
Original IBM PC

The IBM 360 also introduced the idea of having a single “computer architecture” and different implementations of that architecture



Discrete chips with transistors
IBM 360
Discrete chips with logic gates.

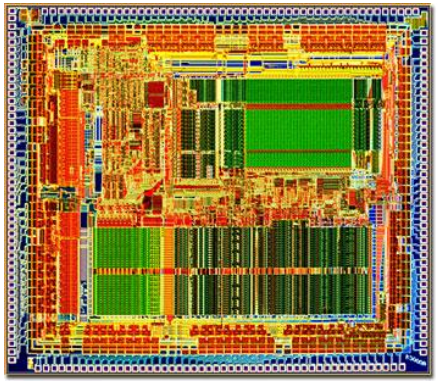
Model	M30	M40	M50	M65
Datapath width	8 bits	16 bits	32 bits	64 bits
Control store size	4k x 50	4k x 52	2.75k x 85	2.75k x 87
Clock rate (ROM cycle time)	1.3 MHz (750 ns)	1.6 MHz (625 ns)	2 MHz (500 ns)	5 MHz (200 ns)
Memory capacity	8–64 KiB	16–256 KiB	64–512 KiB	128–1,024 KiB
Performance (commercial)	29,000 IPS	75,000 IPS	169,000 IPS	567,000 IPS
Performance (scientific)	10,200 IPS	40,000 IPS	133,000 IPS	563,000 IPS
Price (1964 $)	$192,000	$216,000	$460,000	$1,080,000
Price (2018 $)	$1,560,000	$1,760,000	$3,720,000	$8,720,000



Problem with CISC (Complex Instruction Set Computer)

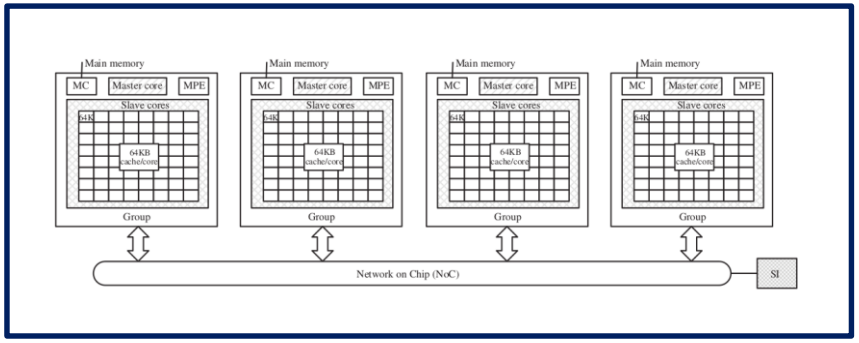
- During the 80s Intel and other manufacturers were driving ever more complex instruction sets since people were programming in assembly and wanted fancier instructions.
- This required writing complex “microprograms” within the hardware chips to execute these instructions.
- Hennessey and Patterson went for a radically new architecture ... RISC (Reduced Instruction Set Computer)

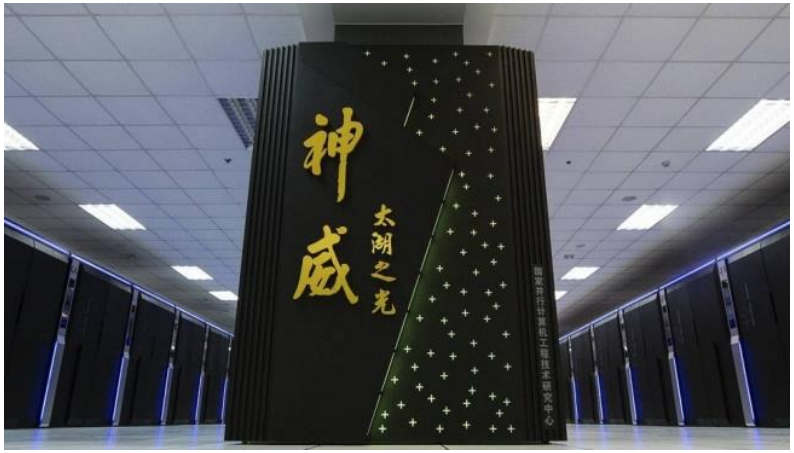
Released:1988 Transistor count: 115,000
This processor was used to render 3D scenes in 90s movies like Jurrasic Park or Terminator 2.



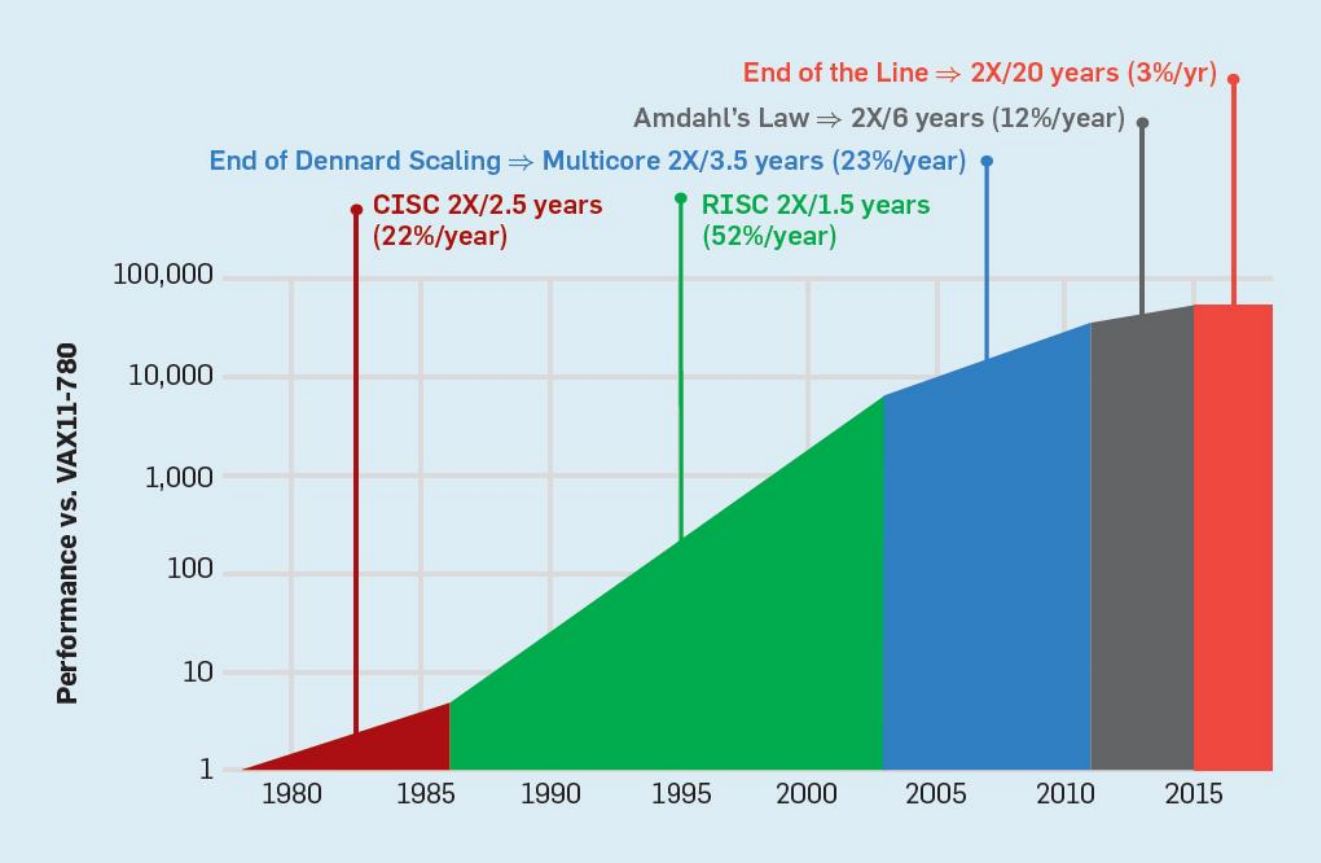
RISC now used for 20 billion portable devices a year … in addition to supercomputers !

Unusually not Intel architecture but custom 260 core 64-bit RISC chips made in China.

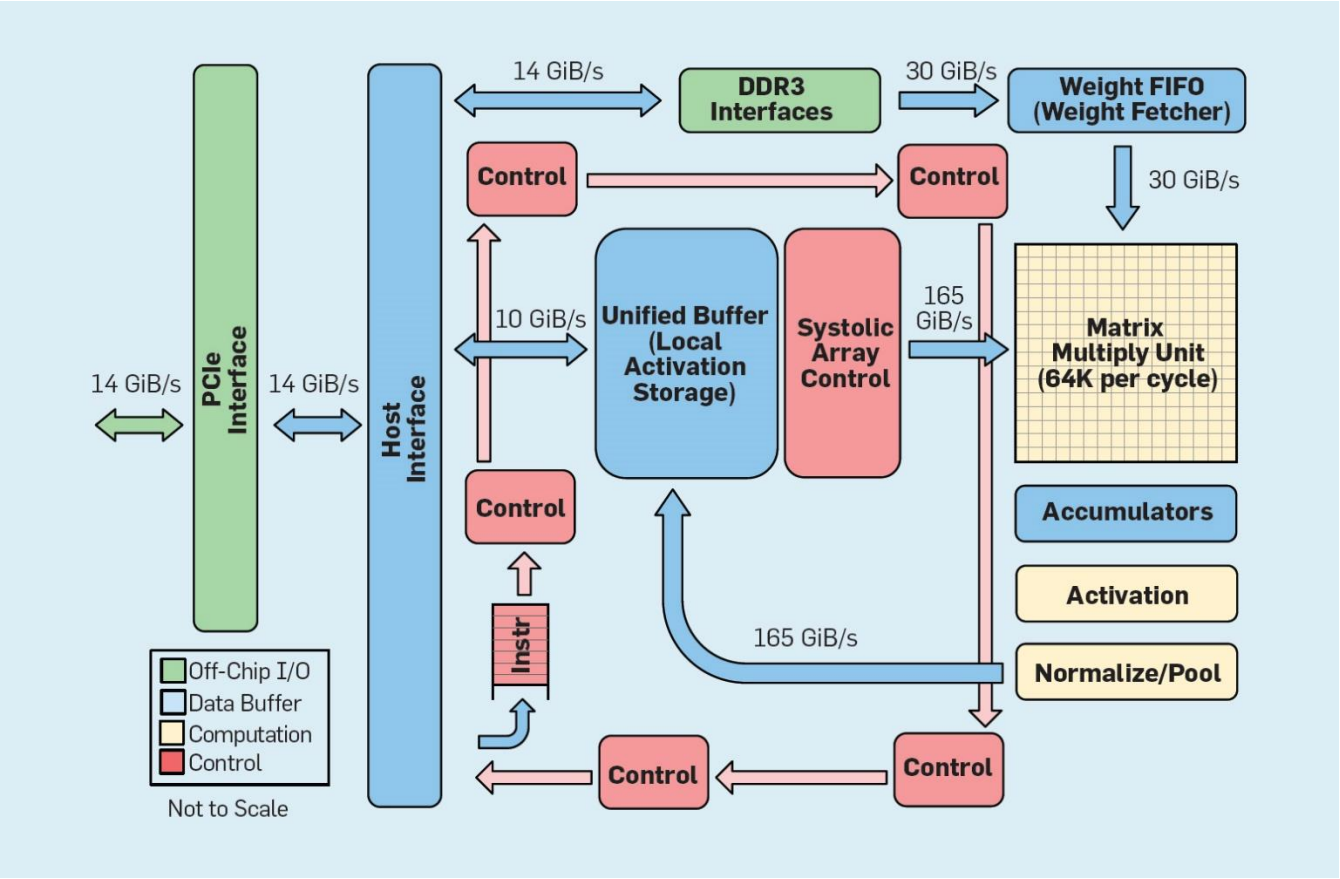




Both hardware and architectures need to evolve to get faster ...



New application specific architectures: Google Tensor Processing Unit (TPU)



Computer architecture continues to evolve ...

IBM Finds Killer App for Tr...

https://www.top500.org/n

Search

Most Visited Getting Started UML use case extend r... Save to Mendeley

Home / News / IBM Finds Killer App for TrueNorth Neuromorphic Chip

IBM Finds Killer App for TrueNorth Neuromorphic Chip

Michael Feldman | September 24, 2016 02:30 CEST

E-mail Tweet Like +1 Share 113

TrueNorth, IBM's brain-like microprocessor, has been found to be exceptionally p
work for deep neural networks. In particular, the chip has demonstrated it's espe
recognition, being able accurately classify such data much more efficiently, from
perspective, than traditional processor architectures, suggesting new application
computing, IoT, robotics, autonomous cars, and HPC.

An IBM Research blog post about TrueNorth's fo
learning, noted that the classification accuracy d
system approached that of state-of-the-art imple
just for image recognition, but for speech recogn
new milestone provides a palpable proof-of-conc
efficiency of brain-inspired computing can be me
effectiveness of deep learning, paving the path to
generation of cognitive computing spanning mob
supercomputers," said IBM Fellow Dharmendra
scientist, Brain-inspired Computing, IBM Research.

D-Wave Sets Stage for 20...

https://www.top500.org/news/d-wave-se

D-wave quantum

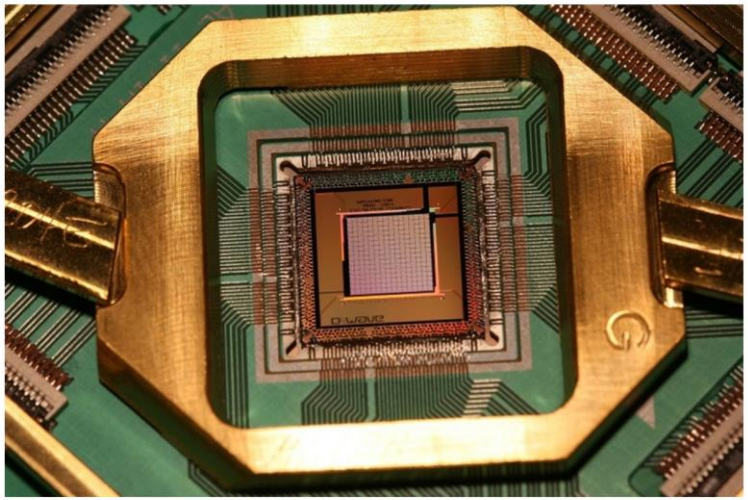
Most Visited Getting Started UML use case extend r... Save to Mendeley Add to My Bookmarks

D-Wave Sets Stage for 2000-Qubit Quantum Computer

Michael Feldman | September 27, 2016 22:28 CEST

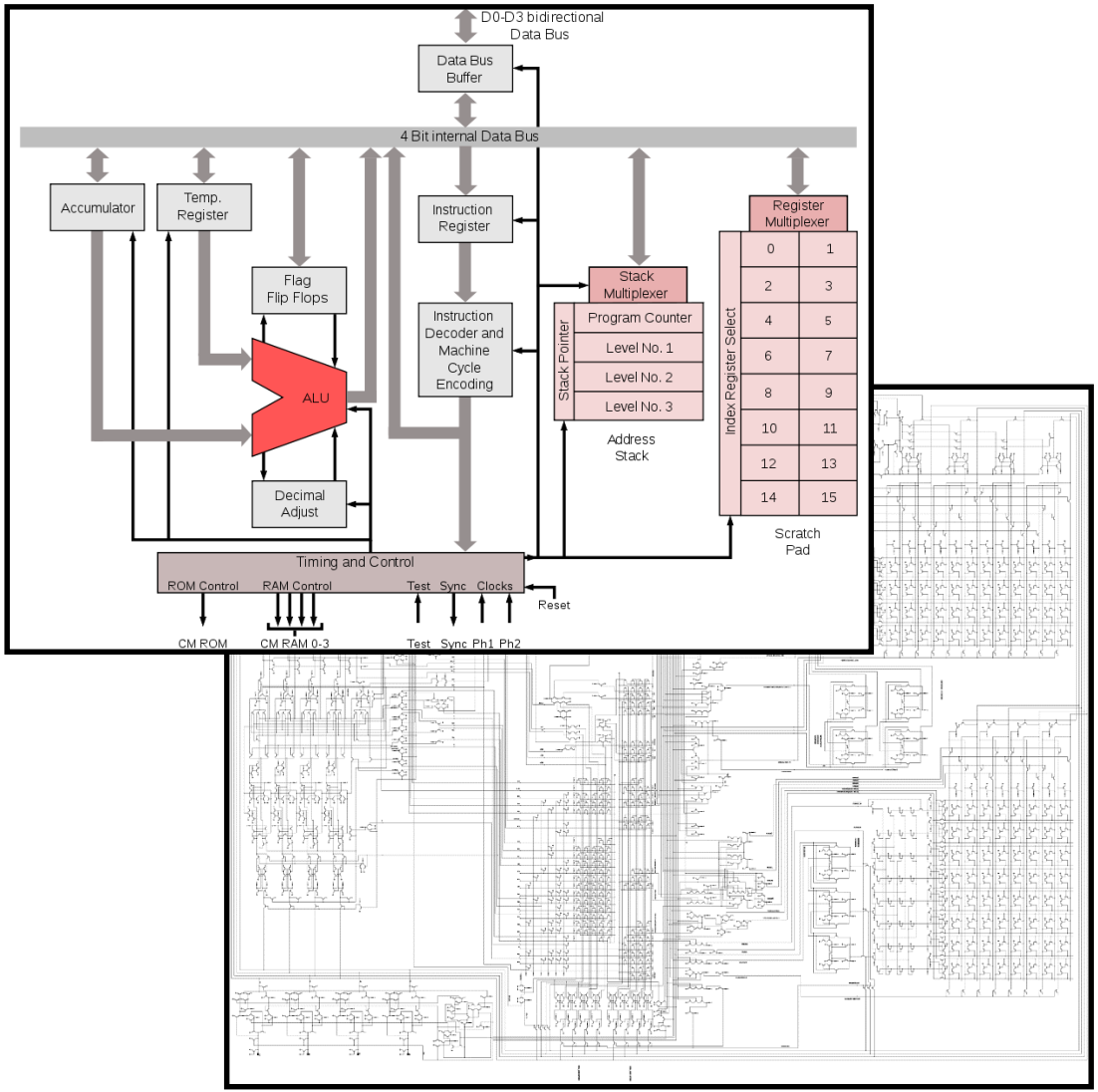
E-mail Tweet Like +1 Share 2

Quantum computing pioneer D-Wave Systems has announced some details of a new 2000-qubit system it has developed. The processor that drives the system will contain twice the number of qubits that powers the current D-Wave 2X system, which was announced last year. The new hardware also includes additional control features that enables users to tune the quantum computations to arrive at a faster result and to explore the solution space more fully.



1000-qubit processor. Image: D-Wave Systems.

Computers can be understood in terms of abstraction layers ... Intel 4004 Processor ...



Application Software	Programs
Operating Systems	Device Drivers
Architecture	Instructions Registers
Micro-architecture	Datapaths Controllers
Logic	Adders Memories
Digital Circuits	AND Gates NOT Gates
Analog Circuits	Amplifiers Filters
Devices	Transistors Diodes
Physics	Electrons

Understanding a modern computer …

- Level 5) Problem Oriented Language Level

```
i = i + 1; // C source
```

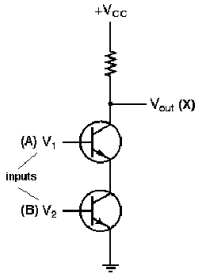
- Level 4) Assembly Language Level
- Level 3) Operating System Level

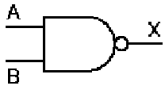
```
add $s3,$s3,1 #  MIPS assembly
```

- Level 2) Instruction Set Architecture Level (ISA Level)

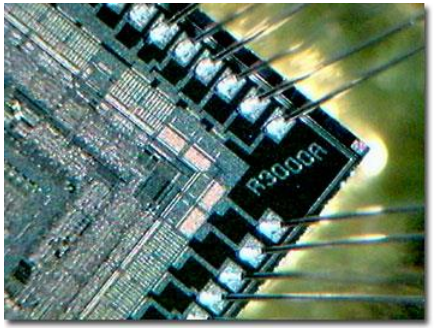
```
001000 01011 01011 0000000000000001
```

- Level 1) Microprogramming Level (not all computers have this level)
- Level 0.5) Modular view (datapath)
- Level 0) Digital Logic Level





A	B	X
0	0	1
0	1	1
1	0	1
1	1	0



Understanding

We will look at how C language constructs are compiled into MIPS assembly and executed.

```
i = i + 1; // C source
```

- Level 5) Problem Oriented Language Level
- Level 4) Assembly

We will learn MIPS assembly language and how machine code works.

- Level 3) Operating System Level

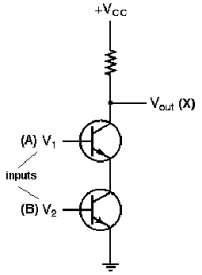
```
add $s3,$s3,1 #  MIPS assembly
```

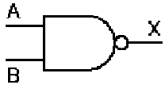
- Level 2) Instruct
  Level (ISA Level)

We will look at computer arithmetic and memory layout since the ISA level is entirely numbers.

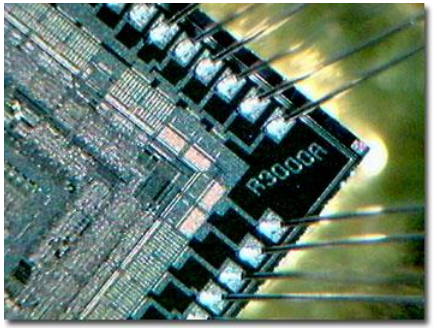
```
001000 01011 01011 0000000000000001
```

- Level 1) Microprogramming Level
  (not all computers have this level)
- Level 0.5) Modular view (datapath)
- Level 0) Digital Logic Level





A	B	X
0	0	1
0	1	1
1	0	1
1	1	0



Why would I want to learn low-level assembly language programming? It is 2020 you know ...

- In the 80s, my little brother was programming complete games in x86 assembly (anyone heard of Lemmings 2 ?)
- He was still using assembly in the 90s ... partly because he was doing 3D graphics engine optimization.
- Working on low-level device drivers then you often want to examine/write assembly language ...
- My cousin is writing a tool-chain (compiler) for an unusual graph-based processor. He says the assembly language is a nightmare!
- Fully understanding concurrency and multithreaded programming requires understanding the machine ...
- Also it allows you to do low-level debugging (you put in print statements and the bug disappears ... what next?)

Summary

- You should be aware that concurrency is fundamental within both hardware and software in computer science ... from the smallest embedded processors to the (un)coordinated vastness of the Internet.
- The aim of this course is to thoroughly understood key principles of Java concurrency so that bugs can be “designed out” of code by analysis rather than detected during testing (often impossible with concurrent code).

- A full understanding of concurrency requires in-depth knowledge of modern computer hardware (cache structure, processor reordering of instructions, etc.)
- Computer architecture and hardware are fundamental topics anyway for computer science (and job interviews!)